SYSTEM DESIGN

Design Instagram

A distributed photo-sharing platform at massive scale

~ 28 minute read

© Study Guide — For personal use only

What's Inside

01 · Problem Statement & Clarifying Questions
02 · Functional & Non-Functional Requirements
03 · Capacity Estimation
04 · High-Level Architecture
05 · Photo Upload Pipeline
06 · Database Design
07 · News Feed Architecture
08 · Feed Ranking & Personalization
09 · Stories Architecture
10 · Search & Explore
11 · Caching Strategy
12 · Notifications
13 · Scalability & Sharding
14 · Failure Modes & Recovery
15 · Trade-offs Analysis
16 · Interview Playbook

01 Problem Statement

Define the scope & constraints

Build a platform enabling users to **upload, share, and discover photos and videos** in real-time with billions of users worldwide.

Key Features

- Users upload photos/videos and compose captions
- Users follow others and see a personalized feed
- Like, comment, and share functionality
- Stories (ephemeral 24-hour content with TTL)
- Search by hashtags, usernames, and captions
- Explore trending content
- Notifications for likes, comments, follows

Clarifying Questions to Ask

Question	Example Answer
Scope?	Photos + videos + Stories? Or just photos for MVP?
Scale?	How many DAU? (Assume 500M)
Geography?	Global or specific regions?
Latency?	Feed load time target? (Assume <2s)
Resolution?	Original + thumbnails for different devices?
Retention?	How long keep photos? (Assume indefinite)
Real-time?	Real-time feed or can delay by minutes?

■ KEY ASSUMPTION

We'll design for **500M DAU, 100M photos/day, global scale, <2s feed latency, and real-time notifications.**

© Study Guide — For personal use only

02 Functional & Non-Functional Requirements

Functional Requirements

- Upload photos/videos with captions and metadata (location, tags)
- Retrieve personalized feed based on follows and engagement
- Like/unlike posts, comment on posts
- Search posts by hashtag, username, caption
- Stories: upload 24-hour ephemeral content
- Follow/unfollow users
- Direct messaging (out of scope for MVP)

Non-Functional Requirements

Requirement	Target
Availability	99.9% uptime (4.3 min/month downtime)
Latency	Feed load <2s, photo upload <5s
Durability	No loss of user data or photos
Consistency	Eventual consistency OK for feed; strong for user profile
Throughput	100M photos/day, 29K feed reads/sec
Scalability	Handle 10x growth without redesign
Cost	Optimize for storage & bandwidth costs
Security	Encryption, auth, rate limiting, abuse prevention

N INSIGHT

Availability vs. Consistency: Feed service can be eventual consistent (stale reads OK), but user account & payment data need strong consistency.

© Study Guide — For personal use only

03 Capacity Estimation

Traffic Estimates

- **500M DAU** — daily active users
- **100M photos/day** — new uploads
- **5 feeds/day per user** — average feeds viewed
- **Feed read QPS:** 500M DAU × 5 / 86,400 sec = **29K queries/sec**
- **Photo upload QPS:** 100M photos / 86,400 = **1.16K uploads/sec**

Storage Estimates

- **Avg photo size:** 200 KB (compressed)
- **Daily raw storage:** 100M photos × 200 KB = **20 TB/day**
- **Thumbnails & variants:** 3x multiplier → **60 TB/day total**
- **Annual storage:** 20 TB × 365 × 3 = **~22 PB/year**
- **5-year retention: ~110 PB total**

✓ STORAGE STRATEGY

Store **originals in S3** (expensive but durable), **thumbnails in CDN edge locations** (fast serving), and **metadata in MySQL** (users, posts) and **Cassandra** (feed timelines, follows).

Bandwidth Estimates

- **Avg photo download size:** 100 KB (compressed + resized)
- **Daily feed views:** 500M × 5 = 2.5B photo loads
- **Outbound bandwidth:** 2.5B × 100 KB = **250 TB/day = 2.3 Gbps**
- **Inbound (uploads):** 100M × 200 KB = **20 TB/day = 185 Mbps**
- **Peak (assume 3x avg): 7 Gbps outbound, 600 Mbps inbound**

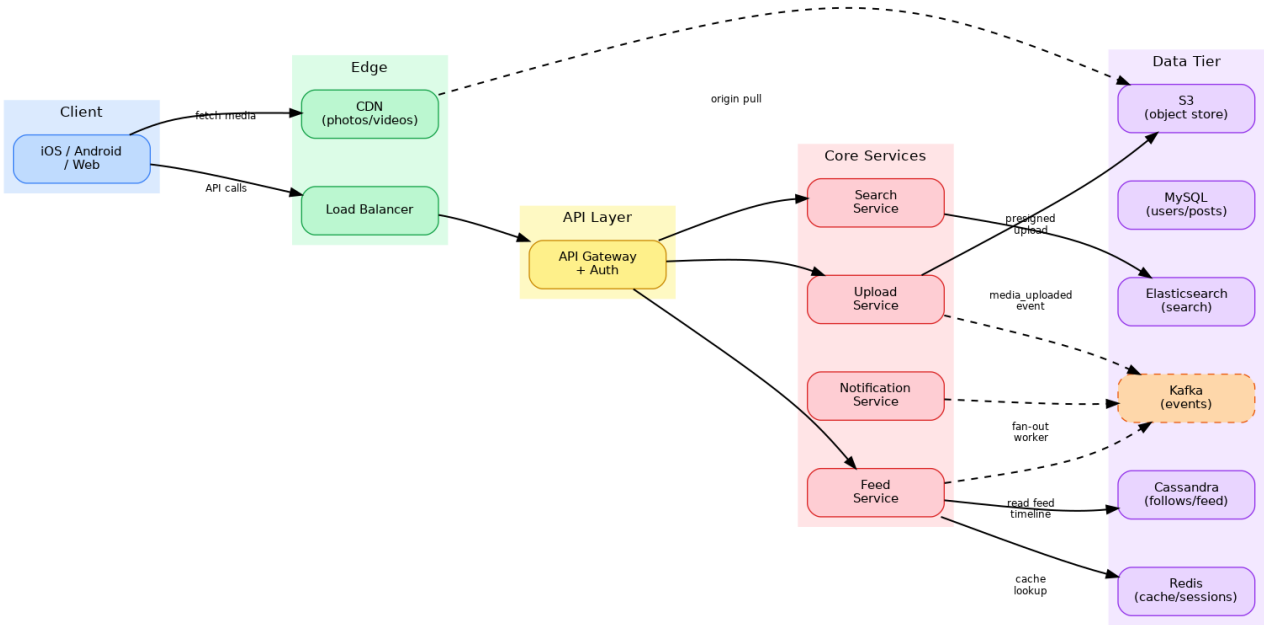
■ KEY NUMBERS TO MEMORIZE

20 TB/day storage, 29K feed reads/sec, 100M photos/day, 500M DAU, 2.3 Gbps bandwidth

© Study Guide — For personal use only

04 High-Level System Architecture

Instagram's architecture follows a **layered service-oriented design**: clients connect through a global load balancer, requests route through an API gateway with authentication, microservices handle core logic (upload, feed, search, notifications), and multiple data stores optimize for their access patterns.



Architecture Layers

- **Client Layer:** iOS, Android, Web browsers with SDKs
- **Edge Layer:** CDN for media delivery, global load balancer for API routing
- **API Gateway:** Authentication, rate limiting, request routing
- **Microservices:** Upload, Feed, Search, Notifications (each independently scalable)
- **Data Tier:** MySQL (strong consistency), Cassandra (eventual consistency), Redis (cache), S3 (object store), Elasticsearch (search), Kafka (event bus)

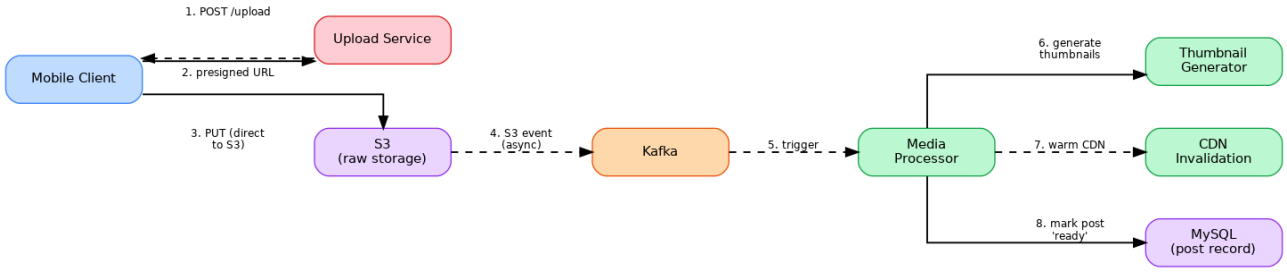
N DESIGN PHILOSOPHY

Each service owns its data store and communicates via **async events** (Kafka) to avoid tight coupling. This enables independent scaling and deployment.

 © Study Guide — For personal use only

05 Photo Upload Pipeline

Uploading a large photo to S3 directly from a mobile client is **slow and error-prone**. The solution: **presigned URLs**. The server generates a time-limited URL allowing the client to PUT directly to S3, bypassing the upload service. A Kafka event triggers async processing.



Step-by-Step Flow

1. Client calls POST /upload to Upload Service with metadata (caption, location, etc.)
2. Service returns a **presigned S3 URL** valid for 15 minutes
3. Client directly PUTs the photo to S3 using presigned URL
4. S3 triggers an async event (S3 → SNS/SQS → Kafka topic)
5. Media Processor service consumes the event and downloads the original from S3
6. Processor generates thumbnails (320×320, 640×640, 1080×1080) and stores variants in S3
7. CDN is pre-warmed with the new photo to reduce latency on first access
8. Database record is marked 'ready', and the post appears in followers' feeds

✓ WHY PRESIGNED URLS?

Offload upload bandwidth from the service tier to clients. Reduces server load by 99%. Also provides **resumable uploads** — if a client disconnects, they retry from where they left off.

 © Study Guide — For personal use only

06 Database Design

Instagram uses **multiple databases optimized for different access patterns**. Strong consistency for user profiles (MySQL), weak consistency for feeds (Cassandra), high throughput counters (Cassandra counters), and fast lookups (Redis cache).

Users Table (MySQL)

```
CREATE TABLE users (
user_id BIGINT PRIMARY KEY,
username VARCHAR(255) UNIQUE NOT NULL,
email VARCHAR(255) UNIQUE NOT NULL,
hashed_password VARCHAR(255),
avatar_url VARCHAR(2048),
bio TEXT,
follower_count INT,
following_count INT,
created_at TIMESTAMP,
updated_at TIMESTAMP
);
```

Sharded by user_id using hash(user_id) % num_shards

Posts Table (MySQL, Sharded)

```
CREATE TABLE posts (
post_id BIGINT PRIMARY KEY,
user_id BIGINT NOT NULL,
caption TEXT,
photo_urls ARRAY<VARCHAR>,
location VARCHAR(255),
hashtags ARRAY<VARCHAR>,
like_count INT,
comment_count INT,
created_at TIMESTAMP
);
CREATE INDEX idx_user_created ON posts(user_id, created_at DESC);
```

Sharded by user_id; enables efficient timeline queries per user

© Study Guide — For personal use only

Follows Table (Cassandra)

```
CREATE TABLE follows (
follower_id BIGINT,
following_id BIGINT,
created_at TIMESTAMP,
PRIMARY KEY (follower_id, following_id)
);
CREATE TABLE reverse_follows (
following_id BIGINT,
follower_id BIGINT,
created_at TIMESTAMP,
PRIMARY KEY (following_id, follower_id)
);
```

Two tables for efficient both directions queries. Partitioned by follower_id

Feed Timeline (Cassandra)

```
CREATE TABLE feed_timeline (
user_id BIGINT,
post_id BIGINT,
post_user_id BIGINT,
created_at TIMESTAMP,
PRIMARY KEY (user_id, created_at DESC, post_id)
);
```

Denormalized feed for each user. Partitioned by user_id, sorted by created_at DESC

■ DENORMALIZATION TRADE-OFF

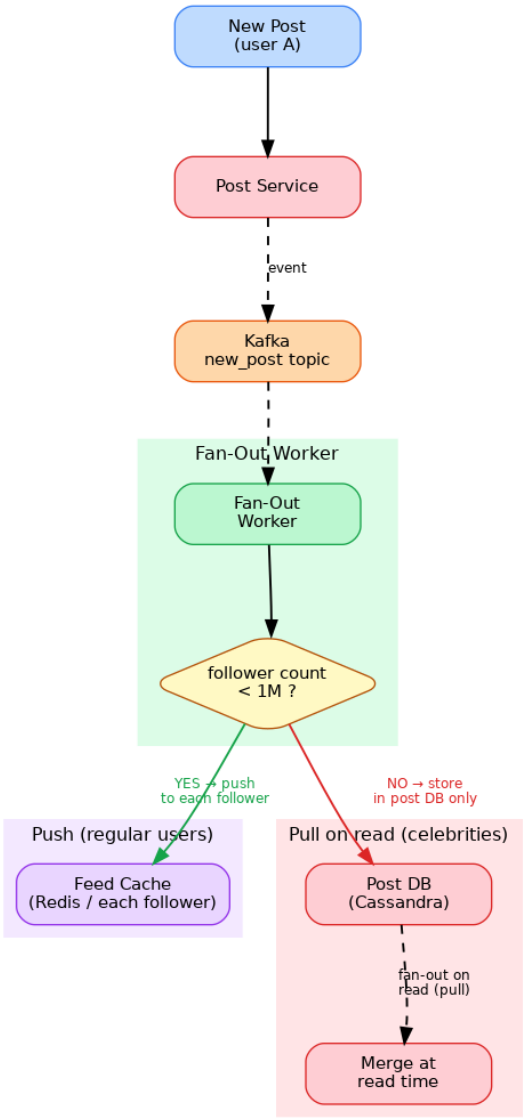
Duplicating posts across users' feeds (write amplification) trades **write cost for read speed**. Feeds return instantly without joining posts table.

© Study Guide — For personal use only

07 News Feed Architecture

Fan-Out: Push vs. Pull

Building a real-time feed for 500M users is **the hardest problem** in Instagram design. Two main approaches exist: **fan-out on write** (push, fast reads) and **fan-out on read** (pull, fast writes). We use a **hybrid approach**.



Approach Comparison

 © Study Guide — For personal use only

Push	Pull	Hybrid (Recommended)
✓ **Fast reads** — feed cached ✗ **Slow writes** — update all followers' caches ✗ **Storage** — duplicate feed per user	✓ **Fast writes** — write once ✗ **Slow reads** — merge on read ✗ **Compute** — expensive merge per request	✓ **Push for regular users** (<1M followers) ✓ **Pull for celebrities** (>1M followers) **Balanced** — 80% write-fast, 20% read-fast

Hybrid Fan-Out Algorithm

1. User A posts a new photo → Post Service publishes event to Kafka **new_post** topic
2. Fan-Out Worker consumes event; queries A's follower count from Cassandra
3. **If follower_count < 1M:** Push to each follower's feed cache in Redis → O(n) writes, O(1) reads
4. **If follower_count >= 1M:** Store post in Cassandra postDB only; skip pushing to individual caches
5. When celebrity's follower requests feed, **fan-out on read:** merge follower's subscriptions with celebrity posts
6. Cache merged feed in Redis for 10 minutes to reduce repeat merges

N WHY HYBRID?

1M threshold is a breakpoint. Regular users (99% of users) get push-based super-fast feeds. Celebrities (0.01% of users) use pull-based efficient writes. Result: **99.9% read latency under 2 sec**.

© Study Guide — For personal use only

08 Feed Ranking & Personalization

Raw chronological feeds are boring. Instagram's **EdgeRank algorithm** ranks posts based on engagement signals, relationship strength, and recency. This keeps users scrolling longer.

Ranking Signals

Signal	Effect
Engagement	Like, comment, share count on the post
Recency	Decay factor: newer posts weighted 1.0, older posts exponentially less
Relationship Strength	How often user interacts with poster (view profile, likes, comments)
Content Type	Video > photo (drives more watch time). Stories > feed posts.
User Preference	ML learns which users, hashtags, locations each user engages with
Time Zone	Boost posts from users in overlapping time zones for synchronous engagement

EdgeRank Formula

```
score = (
(1.0 + engagement_factor) *
recency_decay(time_delta) *
relationship_strength *
content_type_boost
)

engagement_factor = (likes + 2*comments + 3*shares) / max(100, follower_count)
recency_decay(hours) = 1.0 / (1 + 0.5 * hours)
relationship_strength = log(1 + interaction_count) / 10
```

Simplified ranking formula (real Instagram uses 100+ signals)

✓ OFF-LINE TRAINING

ML models are trained offline daily on historical user engagement data. At serve time, the **Feed Service** scores posts in real-time using pre-computed model weights.

© Study Guide — For personal use only

09 Stories Architecture

Ephemeral content with TTL

Stories are temporary (24-hour) photos/videos that disappear after 1 day. They drive significantly higher engagement than feed posts due to urgency and ephemeral FOMO (fear of missing out).

Design Decisions

- **Separate Cassandra table with TTL:** Each story row auto-deletes after 86,400 seconds
- **Partition by user_id, cluster by created_at DESC:** Fast retrieval of user's stories
- **Denormalized data:** Include user metadata (avatar, name) in story row to avoid joins
- **View receipts:** Track which users viewed which stories in a separate counter table
- **Strict order guarantee:** Stories appear in chronological order, no ranking applied
- **CDN caching:** Story media cached for 24 hours; auto-invalidated after TTL expires

Cassandra Schema

```
CREATE TABLE stories (
user_id BIGINT,
story_id BIGINT,
media_url VARCHAR(2048),
caption TEXT,
created_at TIMESTAMP,
PRIMARY KEY (user_id, created_at DESC, story_id)
) WITH CLUSTERING ORDER BY (created_at DESC)
AND default_time_to_live = 86400;
```

Automatic row deletion after 24 hours (86400 seconds)

View Tracking

```
CREATE TABLE story_views (
story_id BIGINT,
viewer_id BIGINT,
viewed_at TIMESTAMP,
PRIMARY KEY (story_id, viewed_at DESC, viewer_id)
);
```


© Study Guide — For personal use only

```
-- Count views per story
CREATE TABLE story_view_counts (
story_id BIGINT PRIMARY KEY,
view_count COUNTER
);
```

Cassandra counters for fast view count increments

■ WHY SEPARATE TABLE?

Stories have different access patterns than feed posts (short TTL, no likes/comments, strict order). Separate storage enables different optimization (TTL, no ranking, no engagement signals).

© Study Guide — For personal use only

10 Search & Explore

Users search for posts by hashtags, captions, and usernames. Instagram's Explore tab surfaces trending content algorithmically. Both features rely on Elasticsearch inverted indices.

Search: Elasticsearch Index

```
PUT /posts
{
"mappings": {
"properties": {
"post_id": { "type": "keyword" },
"user_id": { "type": "keyword" },
"caption": { "type": "text", "analyzer": "standard" },
"hashtags": { "type": "keyword" },
"tags": { "type": "keyword" },
"created_at": { "type": "date" },
"like_count": { "type": "integer" }
}
}
}
```

Elasticsearch mapping for fast full-text search

Search Query Example

```
GET /posts/_search
{
"query": {
"bool": {
"must": [
{ "match": { "caption": "pizza" } }
],
"filter": [
{ "terms": { "hashtags": ["#food", "#nyc"] } },
{ "range": { "created_at": { "gte": "now-30d" } } }
]
```


© Study Guide — For personal use only

```
}
},
"sort": [{ "like_count": "desc" }],
"size": 20
}
```

Search: posts mentioning 'pizza' with #food or #nyc tags in last 30 days

Explore: Trending Hashtags

- **Redis Sorted Set:** Track hashtag popularity in real-time using scores (view count, like count)
- **Time Windows:** Compute trending per region/country, updated every hour
- **Decay:** Older hashtag scores weighted less (e.g., yesterday's hashtags decay 2x)
- **ML Personalization:** Recommend trending hashtags based on user's interests and follower network

```
# Redis trending hashtags (global + region-specific)
ZADD trending_hashtags_global:2024-03-18-10 1500 "#pizza"
ZADD trending_hashtags_global:2024-03-18-10 1200 "#coffee"
ZADD trending_hashtags_us_ny:2024-03-18-10 800 "#nyc"

# Get top 20 trending globally
ZREVRANGE trending_hashtags_global:now 0 19 WITHSCORES

# Personalized: hashtags + user's interests
ZUNIONSTORE user_123_explore 2 trending_hashtags_global user_123:interests
```

Redis commands for trending hashtag leaderboards

© Study Guide — For personal use only

11 Caching Strategy

What, where, and TTL

Caching is critical for performance. Instagram caches user profiles, feed timelines, post metadata, and trending data at multiple layers (client SDK, CDN, Redis).

Caching Layers

Layer	What	Where	TTL	Reasoning
CDN	Photo media (JPG, MP4)	Cloudflare, Akamai edge nodes	7 days	Reduce origin S3 bandwidth by 95%
Redis	User profiles (name, bio, follower count)	Redis cluster (sharded by user_id)	1 hour	Profile updates rare; stale OK for 1h
Redis	Feed timeline (list of post_ids)	Redis cluster	10 minutes	Feed changes every minute; refresh often
Redis	Post metadata (caption, likes, comments)	Redis cluster	30 minutes	Engagement changes frequently
Redis	Trending hashtags	Redis cluster	1 hour	Trends change hourly
Client SDK	Feed (pre-fetched next 20 posts)	Mobile device memory	Session	Reduce API calls when scrolling

© Study Guide — For personal use only

N CACHE INVALIDATION STRATEGIES

TTL-based (simple): Refresh every N minutes. **Event-based (accurate):** When a post is updated, publish event to Kafka; cache service invalidates immediately. **Hybrid:** Use both for redundancy.

Cache Invalidation Code

```
# On post update:
POST /posts/{post_id}
1. Update MySQL/Cassandra
2. Publish 'post_updated' event to Kafka

# Cache service consumes event:
while True:
event = kafka_consumer.poll()
post_id = event['post_id']
redis.delete(f'post_metadata:{post_id}')
redis.delete(f'post_comments:{post_id}')

# Fallback: TTL ensures eventual consistency
# even if event delivery fails
```

Event-based cache invalidation with Kafka

© Study Guide — For personal use only

12 Notifications

Likes, comments, follows, and mentions

Real-time notifications drive user engagement. When someone likes a post, the system must notify the poster within <1 second.

Notification Flow

1. User A likes User B's post → Like Service publishes **post_liked** event to Kafka
2. Notification Service consumes event and queries post metadata (who is the author?)
3. Notification aggregator checks: Did we already notify User B about similar events in the last hour?
4. **Aggregation:** Instead of '**Alice liked your post. Bob liked your post. Carol liked your post.**' → '**Alice, Bob, and 7 others liked your post.**'
5. Deduplication: If User B was already notified 5 minutes ago, skip duplicate
6. Send to **APNs** (iOS) or **FCM** (Android) push notification service
7. APNs/FCM route to user's phone; OS displays notification banner
8. User taps notification → app opens post

Notification Service Architecture

```
class NotificationService:
def on_like_event(event: LikeEvent):
post = db.get_post(event.post_id)
author_id = post.user_id
notif = Notification(
user_id=author_id,
type='like',
actor_ids=[event.user_id], # who liked
post_id=event.post_id
)
# Aggregate with existing notifications
existing = redis.get(f'notif:user:{author_id}')
notif = self.aggregate(notif, existing)
# Send to APNs/FCM
self.send_to_apns(notif)
```

Notification aggregation logic

© Study Guide — For personal use only

■ FILTERING & RATE LIMITING

Users mute notifications from heavy posters. Implement per-user mute lists in Redis. Also rate limit: **max 10 notifications/minute per user** to prevent notification fatigue.

© Study Guide — For personal use only

13 Scalability & Sharding

Horizontal scaling strategies

At 500M DAU, no single database can handle the load. We must **shard data horizontally** across multiple machines.

Sharding Strategy

Table	Sharding Key	# Shards	Reason
users	hash(user_id) % 256	256+	Balance users uniformly; scale independently
posts	hash(user_id) % 256	256+	Co-locate user's posts on same shard (home timeline access)
follows	hash(follower_id) % 256	256+	Fast lookup of who I follow; reverse table for followers
feed_timeline (Cassandra)	hash(user_id) % 1024	1024+	Extremely hot; needs more shards for throughput
comments	hash(post_id) % 512	512+	Group comments per post

■ RESHARDING COMPLEXITY

If we need more shards (e.g., 256 → 512), we must rehash all data. Use **consistent hashing** to minimize impact: only 50% of data moves instead of 100%.

© Study Guide — For personal use only

Read Replicas & Replication

- **MySQL:** 1 primary (writes), 3+ replicas (reads). Replication lag ~100ms
- **Cassandra:** Replication factor 3 (auto-sync across 3 nodes). Eventual consistency.
- **Redis:** Master-slave replicas. AOF persistence to disk.

Connection Pooling

```
# Connection pool to MySQL shard
pool = MySQLConnectionPool(
pool_size=50, # max connections per shard
max_retries=3,
retry_delay=0.1
)

# Usage
shard_id = hash(user_id) % 256
conn = pool.get_connection(shard_id)
result = conn.execute(query)
conn.return_to_pool()
```

Connection pooling reduces latency and prevents connection exhaustion

© Study Guide — For personal use only

14 Failure Modes & Recovery

Distributed systems fail constantly. We must design for graceful degradation and recovery.

Failure Scenarios

Failure Mode	Mitigation Strategy
Database shard down	Route traffic to replica. If replica also down, return error + cached data. SLA: 99.9%
CDN edge node down	DNS failover to backup CDN. Origin S3 handles spike. Latency increases 50ms.
API Gateway failure	Multiple gateways behind auto-scaling. One failure = others handle traffic. Scaling: 2–3 min
Kafka broker down	Kafka is replicated across 3+ brokers. Automatic failover. Messages buffered up to 7 days.
Elasticsearch cluster down	Search unavailable but core features (feed, upload) unaffected. Cache trending queries. Recovery: 30 min
Network partition (split brain)	Cassandra uses quorum reads/writes. 2/3 nodes must agree. Consistency over availability.
Cascading failure	Circuit breaker pattern: if Feed Service is slow, API Gateway returns cached feed instead of failing.
Data corruption	Versioned backups taken every 6 hours. Can restore to any point in time within 7 days.

N CIRCUIT BREAKER PATTERN

Prevent cascading failures: **if downstream service is slow (SLA violated), return cached response instead of waiting/retrying.** Allows damaged system to recover.

© Study Guide — For personal use only

15 Trade-offs Analysis

Key architectural decisions explained

• **Consistency vs. Availability:** Feed is eventually consistent (allows stale reads). User profile is strongly consistent (always fresh). Real Instagram is consistent for critical data.

•

• **Write vs. Read Latency:** Push-based fan-out (write slow) optimizes for read speed. Pull-based (read slow) optimizes for write speed. Hybrid balances both.

•

• **Storage vs. Compute:** Denormalized feed table (storage waste) enables O(1) feed reads. Normalized schema (storage efficient) requires expensive joins.

•

• **Fresh Data vs. Cache Hits:** Short TTL (1h) keeps data fresh but increases cache misses → database load. Long TTL (24h) hits cache but serves stale data. Sweet spot: 1h for user profiles, 10min for feeds.

•

• **Monolith vs. Microservices:** Single codebase (simpler deployment) vs. services (independent scaling, fault isolation). Instagram chose services at scale.

•

• **SQL vs. NoSQL:** MySQL (transactions, SQL queries) for strong consistency. Cassandra (distributed, high throughput) for eventual consistency. Both needed.

N NO SILVER BULLET

Every system design is a set of trade-offs. There's no perfect architecture. Know your constraints (scale, consistency, latency) and choose the right trade-off for each component.

© Study Guide — For personal use only

16 Interview Playbook

45-minute execution strategy

45 minutes is tight. You must cover architecture, trade-offs, and scalability. This playbook guides you through each phase.

00:00–02:00 — Clarify Requirements

- Ask: 'Should I focus on photos, videos, or both?' (Answer: assume photos for MVP)
- Ask: 'Scale? Assume 500M DAU?' (Confirming assumptions avoids derailing)
- Ask: 'Real-time feed or eventual consistent?' (Answer: real-time <2s)
- Ask: 'Do we need messaging?' (Answer: out of scope, core features only)

02:00–07:00 — Capacity Estimation

- Always estimate before designing.
- **Memorize:** 500M DAU, 100M photos/day, 20TB/day storage, 29K feed reads/sec, 2.3 Gbps bandwidth
- Show the math: '500M × 5 feeds / 86400s = 29K qps'
- Interviewer will validate your numbers.

07:00–20:00 — High-Level Architecture

- Draw boxes: client → CDN → LB → API Gateway → services → databases
- Name the key services: Upload, Feed, Search, Notifications
- Mention Kafka for async events (decoupling)
- Name data stores: MySQL (strong), Cassandra (eventual), Redis (cache), S3 (objects), ES (search)
- Don't dive into details yet. Focus on interactions.

20:00–35:00 — Deep Dive into Feed (Hardest Problem)

- Discuss push vs. pull problem: 'If celebrity posts, we can't update 100M followers' caches (too slow).'
- Explain hybrid: push for <1M followers, pull for celebrities
- Describe fan-out worker, Kafka event, feed cache in Redis
- Mention denormalized feed table in Cassandra
- Interviewer will ask: 'What if a celebrity's follower count crosses 1M?' Answer: 'Use a threshold service to toggle.'

35:00–40:00 — Address Remaining Components

- Upload: presigned URLs, async media processor

© Study Guide — For personal use only

- Search: Elasticsearch inverted indices
- Notifications: aggregation + APNs/FCM

40:00–45:00 — Trade-offs & Recovery

- Mention: 'We chose eventual consistency for feed to trade write latency for read speed.'
- Failure: 'If Cassandra shard fails, serve from cache. Consistency recovers when shard is back.'
- Scaling: 'Shard by user_id. If adding shards, use consistent hashing.'

Key Numbers to Memorize (for stress test)

```
500M DAU
100M photos/day (1.16K photos/sec upload)
20 TB/day raw storage (60 TB with variants)
29K feed read QPS (500M × 5 / 86400)
2.3 Gbps outbound bandwidth (7 Gbps peak)
10 Gbps inbound bandwidth (uploads + metadata)
```

Recite these to show confidence

✓ IF YOU GET STUCK

Interviewer will guide you. **Don't panic.** Ask clarifying questions. 'If this assumption is wrong, it changes the design. Is that correct?' Shows thoughtful thinking.

© Study Guide — For personal use only